Clase 8 (Unidad 3) — Seguridad de APIs: OWASP, rate limiting y CORS

Duración total: 4 horas | Proyecto: throttling, CORS, sanitización y auditoría

Cómo leer esta guía

Cada parte combina dos tipos de contenido:

- **Conceptos** (qué es y por qué): la base teórica de la Unidad 3, con tablas y diagramas. Se pueden leer antes de la clase.
- **Práctica** (cómo): los pasos con código, que aplican esos conceptos a TaskFlow.

Al final hay **preguntas de repaso** para autoevaluarse.

Parte 0 — Antes de empezar (5 min)

Parados en `taskflow/`, entorno virtual activo. Con membresías y permisos ya funcionando (Clase 7), hoy cerramos la Unidad 3 revisando la seguridad de la API en conjunto, no feature por feature.

Parte 1 — OWASP Top 10 aplicado a TaskFlow (60 min)

1.1 ¿Qué es el OWASP Top 10?

Es un listado, actualizado periódicamente por la fundación **OWASP** (Open Worldwide Application Security Project), de los riesgos de seguridad más comunes en aplicaciones web. No es específico de ningún lenguaje ni framework: aplica igual a un backend en Django, en Node o en cualquier otro stack. Repasamos los ítems que más nos tocan hasta ahora, con ejemplos concretos ya resueltos (o pendientes) en TaskFlow:

- **Broken Access Control** (control de acceso roto): lo resolvimos en las Clases 6 y 7 filtrando `get_queryset` por membresía al proyecto y verificando el rol en `perform_create`. Si no lo hubiéramos hecho, cualquier usuario autenticado podría leer tareas ajenas simplemente cambiando el número de `id` en la URL.
- **Cryptographic Failures** (fallas criptográficas): nunca guardar contraseñas en texto plano. Ya lo resolvimos en la Clase 6 usando `create_user`, que las hashea automáticamente con un algoritmo seguro (PBKDF2 por defecto en Django).
- **Injection** (inyección): el ORM de Django parametriza las consultas SQL automáticamente, lo que nos protege de SQL Injection **mientras no escribamos SQL crudo concatenando texto recibido del usuario** (algo que en toda la materia evitamos a propósito, usando siempre el ORM). Hoy agregamos la otra cara: la inyección de HTML/JavaScript (XSS), Parte 3.2.
- **Security Misconfiguration** (configuración de seguridad incorrecta): `DEBUG=True` en producción (muestra detalles internos del error a cualquier visitante), `SECRET_KEY` expuesta en el repositorio (evitado desde la Clase 1 con `.env`), políticas CORS abiertas a cualquier origen (Parte 3.1).
- **Identification and Authentication Failures** (fallas de identificación y autenticación): tokens sin expiración razonable (ya limitado en la Clase 6), contraseñas débiles (validadas en el registro de la Clase 6), ausencia de límite de intentos en el login (lo resolvemos hoy, Parte 2).
- **Security Logging and Monitoring Failures** (fallas de registro y monitoreo): si nadie registra los intentos de login fallidos, un ataque puede pasar inadvertido. Lo resolvemos hoy, Parte 3.3.

> 💡 **Nota del docente**
> Este repaso no reemplaza la lectura del documento oficial de OWASP (disponible gratuitamente en owasp.org); es el punto de partida para que reconozcan estos riesgos cuando aparezcan en cualquier stack, no solo en Django. La lista se actualiza cada pocos años: verifiquen en el sitio cuál es la edición vigente.

Concepto: pensar como atacante. Asegurar una aplicación empieza por tres preguntas, que en conjunto se llaman **modelo de amenazas**:

- **¿Qué hay que proteger?** (los **activos**): en TaskFlow, las cuentas de usuario, los datos de los proyectos, la `SECRET_KEY`, la disponibilidad del servicio.

- **¿Por dónde pueden entrar?** (la **superficie de ataque**): cada endpoint, cada formulario, cada header que leemos, cada dependencia instalada. Cada funcionalidad nueva la agranda; por eso se cierra todo por defecto (Clase 6).
- **¿Quién podría atacar y cómo?** (las **amenazas**): un usuario curioso que cambia un `id` en la URL, un bot que prueba contraseñas, una página maliciosa que usa el navegador de nuestros usuarios.

Concepto: defensa en profundidad. Ninguna medida es perfecta, así que se apilan varias capas independientes: si una falla, la siguiente frena el ataque. Así quedan las capas de TaskFlow al terminar esta clase:

```
Petición
  ▼ HTTPS (plataforma, Clase 12)      → nadie lee ni altera el tráfico
  ▼ CORS (hoy)                        → otros sitios no usan el navegador
  ▼ Rate limiting (hoy)               → frena fuerza bruta y abuso
  ▼ JWT (Clase 6)                     → sabemos quién es
  ▼ Permisos y get_queryset (Cl. 7)   → ve y hace solo lo suyo
  ▼ Validación y sanitización         → los datos entran limpios
  ▼ ORM parametrizado (Clase 3)       → sin inyección SQL
  ▼ Restricciones de la base (Cl. 7)  → integridad garantizada
  ▼ Auditoría y logs (hoy)            → todo queda registrado
```

Concepto: la inyección, con un ejemplo. Una inyección ocurre cuando un dato del usuario se mezcla con código que se ejecuta. Si armáramos SQL concatenando texto:

```
# MAL: nunca hagan esto
titulo = request.GET["q"]      # el usuario manda:  x' OR '1'='1
sql = f"SELECT * FROM core_task WHERE title = '{titulo}'"

# SQL que se ejecuta (devuelve TODAS las tareas):
# SELECT * FROM core_task WHERE title = 'x' OR '1'='1'
```

El ORM, en cambio, manda el SQL y los datos **por separado** (consultas **parametrizadas**): `Task.objects.filter(title=titulo)` trata el texto del usuario siempre como un valor, nunca como código. La misma idea se aplica a comandos del sistema operativo, a consultas de MongoDB y a los prompts de una IA (la llamada *prompt injection*).

1.2 Ejercicio: auditoría del propio proyecto

En parejas, abran su propio proyecto `taskflow/` y repasen, ítem por ítem de la lista anterior, si está cubierto o no, anotando en un documento de texto compartido. Algunas preguntas concretas para guiarse:

- ¿Qué pasa si pido `GET /api/tasks/<id>/` de una tarea de un proyecto donde no soy miembro? ¿Y si soy miembro pero hago `DELETE`?
- ¿`.env` aparece en `git status`? ¿Alguna vez se subió al repositorio? (Si se subió, cambiar la `SECRET_KEY` es obligatorio: borrarlo después no alcanza, sigue en el historial de Git.)
- ¿Cuántas veces seguidas puedo probar contraseñas en `/api/token/`?

Pónganlo en común con el resto del curso al final de esta parte.

Parte 2 — Rate limiting (throttling) (55 min)

2.1 Concepto: por qué y cómo limitar peticiones

Sin límite de peticiones, cualquiera puede intentar miles de contraseñas por segundo contra `/api/token/` (un ataque de **fuerza bruta**), o simplemente saturar el servidor con peticiones legítimas pero excesivas, degradando el servicio para todos los demás.

Los abusos más comunes que frena un límite de peticiones:

Ataque	En qué consiste	Qué lo frena
Fuerza bruta	Probar muchas contraseñas contra **una** cuenta	Límite estricto en el login, por IP y por usuario
Relleno de credenciales (*credential stuffing*)	Probar usuarios y contraseñas filtrados de **otros** sitios, confiando en que la gente repite contraseñas	Límite en el login y contraseñas únicas (validadas en el registro, Clase 6)

Denegación de servicio (DoS)	Saturar el servidor con peticiones para que no atienda a nadie	Límites generales; a gran escala, protección en la red o en la plataforma
Extracción masiva (*scraping*)	Descargar todos los datos de a poco	Límites por usuario y paginación (Clase 5)

Existen varios algoritmos para contar peticiones:

Algoritmo	Cómo cuenta	Característica
Ventana fija	Cuenta las peticiones de cada minuto "de reloj" (de 10:00:00 a 10:00:59)	Simple, pero permite el doble de peticiones en el borde entre dos ventanas
Ventana deslizante	Guarda la hora de cada petición y cuenta las de los últimos 60 segundos	Precisa. **Es la que usa DRF**
Balde de fichas (*token bucket*)	Un balde se recarga a ritmo constante y cada petición gasta una ficha	Permite ráfagas cortas; muy usado en gateways y en la nube

Cuando se supera el límite, la respuesta estándar es `429 Too Many Requests`, con el header `Retry-After` que indica cuántos segundos esperar. Un cliente bien hecho lo respeta en vez de reintentar enseguida.

> 💡 **Nota del docente**
> ¿Por qué no bloquear la cuenta después de 5 intentos fallidos? Porque un atacante podría bloquear a propósito la cuenta de cualquier usuario con solo equivocarse 5 veces: sería una denegación de servicio contra esa persona. Por eso se prefiere limitar el ritmo (5 por minuto) antes que bloquear.

2.2 Configuración con DRF

En `taskflow/config/settings.py`, dentro del diccionario `REST_FRAMEWORK` ya existente, agreguen al final:

```
REST_FRAMEWORK = {
    # ... todo lo que ya tenían de clases anteriores
    "DEFAULT_THROTTLE_CLASSES": [
        "rest_framework.throttling.AnonRateThrottle",
        "rest_framework.throttling.UserRateThrottle",
    ],
    "DEFAULT_THROTTLE_RATES": {
        "anon": "20/minute",
        "user": "100/minute",
        "login": "5/minute",
    },
}
```

- `AnonRateThrottle` limita peticiones de usuarios **no autenticados**, identificándolos por su dirección IP.
- `UserRateThrottle` limita peticiones de usuarios **autenticados**, identificándolos por su `id` de usuario.
- `DEFAULT_THROTTLE_RATES` define cuántas peticiones se permiten por unidad de tiempo para cada tipo: 20 por minuto para anónimos, 100 por minuto para autenticados, y 5 por minuto para el login (lo usamos enseguida).
- Cuando se supera el límite, DRF responde `429 Too Many Requests` y agrega el header `Retry-After` con los segundos que hay que esperar.

Para el endpoint de login (`/api/token/`), que es especialmente sensible frente a fuerza bruta, conviene un límite mucho más estricto y propio. Creen el archivo `taskflow/core/throttles.py`:

```
from rest_framework.throttling import SimpleRateThrottle


class LoginRateThrottle(SimpleRateThrottle):
    scope = "login"

    def get_cache_key(self, request, view):
        ip = self.get_ident(request)
```

```
        return self.cache_format % {"scope": self.scope, "ident": ip}
```

- `scope = "login"` conecta esta clase con la entrada `"login": "5/minute"` de `DEFAULT_THROTTLE_RATES`.
- `get_cache_key` define **a quién** se le cuenta el límite: acá, a cada IP (`self.get_ident(request)`), porque en el login todavía no hay un usuario autenticado al cual asociarlo.
- El contador vive en la **caché** de Django (la misma de los feriados de la Clase 5). Por eso, si reinician el servidor, los contadores vuelven a cero.

2.3 Aplicarlo al login y registrar los intentos fallidos

Para aplicar este throttle a la vista de login (que viene de `simplejwt`, no la escribimos nosotros), la **heredamos** en una vista propia. Aprovechamos para dejar registrado cada intento fallido; el modelo `SecurityLog` que usamos acá lo creamos en la Parte 3.3. En `taskflow/core/views.py`:

```
import logging

from rest_framework.exceptions import AuthenticationFailed, PermissionDenied
from rest_framework_simplejwt.views import TokenObtainPairView

from .models import Project, ProjectMembership, SecurityLog
from .throttles import LoginRateThrottle

security_logger = logging.getLogger("security")


class ThrottledTokenObtainPairView(TokenObtainPairView):
    throttle_classes = [LoginRateThrottle]

    def post(self, request, *args, **kwargs):
        try:
            return super().post(request, *args, **kwargs)
        except AuthenticationFailed:
            ip = request.META.get("REMOTE_ADDR")
            username = request.data.get("username", "")
            SecurityLog.objects.create(action="login_failed", detail=f"usuario={username}",
ip_address=ip)
            security_logger.warning(f"Login fallido para usuario {username} desde {ip}")
            raise
```

- `throttle_classes = [LoginRateThrottle]` reemplaza, **solo para esta vista**, los throttles globales por el nuestro.
- `super().post(...)` ejecuta el login original de `simplejwt`. Si las credenciales son incorrectas, `simplejwt` lanza `AuthenticationFailed`.
- La atrapamos con `except` solo para registrar el evento, y con `raise` (sin nada después) la volvemos a lanzar tal cual: el cliente sigue recibiendo el mismo `401` de siempre.
- `request.META.get("REMOTE_ADDR")` es la IP de quien hizo la petición.

En `taskflow/config/urls.py`, reemplacen la vista usada para `/api/token/` (y su import):

```
from rest_framework_simplejwt.views import TokenRefreshView
from core.views import RegisterView, SocialLoginJWTView, ThrottledTokenObtainPairView

urlpatterns = [
    # ...
    path("api/token/", ThrottledTokenObtainPairView.as_view(), name="token_obtain_pair"),
    path("api/token/refresh/", TokenRefreshView.as_view(), name="token_refresh"),
    # ...
]
```

La prueba la hacemos al final de la Parte 3.3. Hasta que creemos el modelo `SecurityLog`, el servidor de desarrollo va a mostrar un `ImportError` en la consola: es esperable, no se asusten, se resuelve solo al completar la Parte 3.3.

Parte 3 — CORS, sanitización y auditoría (70 min)

3.1 Políticas CORS

Concepto: la política del mismo origen. Los navegadores aplican una regla de seguridad fundamental, la **política del mismo origen** (*Same-Origin Policy*): el JavaScript de una página solo puede leer respuestas de su **mismo origen**. El origen es la combinación de **protocolo + dominio + puerto**:

Página en	Pide a	¿Mismo origen?
`http://localhost:3000`	`http://localhost:3000/api/`	Sí
`http://localhost:3000`	`http://localhost:8000/api/`	No: cambia el puerto
`http://taskflow-app.com`	`https://taskflow-app.com/api/`	No: cambia el protocolo
`https://taskflow-app.com`	`https://api.taskflow-app.com/`	No: cambia el dominio

Sin esa regla, cualquier sitio que visitemos podría leer, con nuestro navegador, los datos de las otras páginas donde estamos logueados. **CORS** es la forma **controlada** de hacer excepciones: el servidor declara, con headers, qué otros orígenes pueden leer sus respuestas.

Cuando la petición no es "simple" (por ejemplo, usa el header `Authorization` o manda JSON, como todas las de nuestra API), el navegador primero hace una **petición previa** (*preflight*) con el método `OPTIONS`, para preguntar si está permitida. Esta es la respuesta real de TaskFlow a una petición previa desde `http://localhost:3000`:

```
OPTIONS /api/tasks/
Origin: http://localhost:3000
Access-Control-Request-Method: POST
Access-Control-Request-Headers: authorization, content-type

→ 200 OK
access-control-allow-origin: http://localhost:3000
access-control-allow-methods: DELETE, GET, OPTIONS, PATCH, POST, PUT
access-control-allow-headers: accept, authorization, content-type, user-agent, x-csrftoken, x-requested-
with
access-control-max-age: 86400
```

Si el origen no está permitido, el servidor responde **sin** esos headers y el navegador no hace la petición real. `access-control-max-age` indica cuántos segundos puede el navegador recordar el permiso sin volver a preguntar (acá, un día).

Concepto: CORS no es CSRF. Son dos cosas distintas, y se confunden seguido:

- **CORS** decide si otro sitio puede **leer** las respuestas de nuestra API desde el navegador.
- **CSRF** (*Cross-Site Request Forgery*, falsificación de petición entre sitios) es un ataque en el que otro sitio hace que el navegador del usuario **envíe** una petición a nuestra API, aprovechando que el navegador adjunta las cookies de sesión automáticamente. Django lo previene con `CsrfViewMiddleware` (un token que el sitio malicioso no puede conocer). Nuestros endpoints con JWT no son vulnerables a CSRF, porque el header `Authorization` no se adjunta solo: lo agrega el código de la página.

Práctica.

CORS (Cross-Origin Resource Sharing) es un mecanismo de seguridad implementado por el **navegador** (no por nuestro backend en sí, aunque nuestro backend es quien lo configura) que restringe qué **orígenes** (combinación de protocolo + dominio + puerto) pueden hacer peticiones JavaScript hacia nuestra API desde el navegador de un usuario. Si en el futuro un frontend alojado en `https://taskflow-app.com` va a consumir esta API, tenemos que autorizarlo explícitamente.

Importante: CORS **no** protege la API de `curl`, Postman o un script de Python, que no son navegadores y no lo respetan. Protege a **nuestros usuarios**: impide que una página maliciosa que visiten use su navegador para hablar con nuestra API.

```
pip install django-cors-headers
pip freeze > requirements.txt
```

En `taskflow/config/settings.py`, agreguen la línea de `CorsMiddleware` **al principio** de la lista `MIDDLEWARE` que ya existe (no reemplacen la lista entera):

```
MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",  # nueva: tiene que ir primera
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
    "django.middleware.clickjacking.XFrameOptionsMiddleware",
]
```

Y al final del archivo:

```
# --- Seguridad (Clase 8) ---
INSTALLED_APPS += ["corsheaders"]

CORS_ALLOWED_ORIGINS = [
    "http://localhost:3000",
    "https://taskflow-app.com",
]
```

💡 **Nota del docente**
`MIDDLEWARE` es una lista **ordenada**: cada petición la atraviesa de arriba hacia abajo antes de llegar a la vista, y la respuesta la atraviesa de abajo hacia arriba antes de salir. `CorsMiddleware` tiene que ir lo más arriba posible, y siempre **antes** de `CommonMiddleware`, para poder agregar los headers de CORS a todas las respuestas, incluso a las que genere otro middleware. No se preocupen por el `AccountMiddleware` de `allauth` (Clase 7): lo agregamos con `MIDDLEWARE += [...]` al final del archivo y sigue ahí.

💡 **Nota del docente**
Nunca usen `CORS_ALLOW_ALL_ORIGINS = True` en producción: equivale a decirle a cualquier sitio web del mundo que puede hacer peticiones contra nuestra API desde el navegador de sus visitantes.

Para ver CORS funcionando sin un frontend, simulen con `curl` el header `Origin` que mandaría un navegador (usen un token válido):

```
curl -i http://127.0.0.1:8000/api/tasks/ -H "Origin: http://localhost:3000" -H "Authorization: Bearer <TOKEN>"
curl -i http://127.0.0.1:8000/api/tasks/ -H "Origin: http://sitio-malicioso.com" -H "Authorization: Bearer <TOKEN>"
```

En la primera respuesta aparece el header `access-control-allow-origin: http://localhost:3000`; en la segunda, no. Sin ese header, el navegador le bloquea la respuesta al JavaScript de la página maliciosa.

3.2 Sanitización de entradas

Concepto: XSS. El *Cross-Site Scripting* consiste en lograr que el navegador de **otro** usuario ejecute código JavaScript del atacante, que así puede robar su token o actuar en su nombre. Hay tres variantes:

Tipo	Cómo ocurre	Ejemplo en TaskFlow
Almacenado	El código se guarda en la base y se ejecuta cada vez que alguien ve el dato	Una descripción de tarea con `<script>`, que ven todos los miembros del proyecto
Reflejado	El código viaja en la URL y la página lo muestra tal cual	Una página de búsqueda que muestra "Resultados para: `<script>...`"
Basado en el DOM	El JavaScript de la página inserta datos con `innerHTML` sin limpiarlos	Un frontend que arma la lista de tareas con `innerHTML`

Concepto: validar, sanitizar y escapar. Son tres defensas distintas, que se complementan:

- **Validar:** rechazar lo que no cumple las reglas (Clase 4). La entrada se acepta o no.
- **Sanitizar:** limpiar la entrada, quitando lo peligroso (lo que hacemos hoy con `nh3`). La entrada se acepta, modificada.
- **Escapar** (codificar la salida): al **mostrar** el dato, convertir `<` en `<` para que el navegador lo muestre como texto y no lo ejecute. Es la defensa principal contra XSS, y le corresponde al frontend: los templates de Django lo hacen solos, y en JavaScript se logra usando `textContent` en vez de `innerHTML` (Clase 13).

Sanitizar en el backend protege a **todos** los clientes, incluso a los que se olvidan de escapar. Por eso `nh3` trabaja con una **lista blanca** (*allowlist*): se define lo que **se permite** (acá, ninguna etiqueta) en vez de intentar enumerar todo lo peligroso, que es imposible.

Práctica.

DRF y el ORM ya nos protegen de la inyección SQL, pero cuando recibimos texto libre que eventualmente se va a **mostrar** en algún lado (por ejemplo, la `description` de una tarea en un futuro frontend), conviene evitar que contenga HTML o JavaScript malicioso. Es el ataque conocido como **XSS** (*Cross-Site Scripting*).

Usamos `nh3`, una librería gratuita y de código abierto especializada en limpiar HTML:

```
pip install nh3
pip freeze > requirements.txt
```

> 💡 **Nota del docente**
> Durante años la librería estándar para esto fue `bleach`, y la van a encontrar en muchos tutoriales. Sus propios autores la declararon **discontinuada** en 2023 y recomiendan `nh3` como reemplazo. Para proyectos nuevos, usen `nh3`.

En `taskflow/core/serializers.py`, agreguen el import arriba y un validador para `description` dentro de `TaskSerializer`:

```
import nh3


class TaskSerializer(serializers.ModelSerializer):
    # ... lo que ya tenían

    def validate_description(self, value):
        return nh3.clean(value, tags=set())
```

`validate_description` sigue el mismo patrón `validate_<campo>` de la Clase 4, pero en vez de rechazar el valor, lo **limpia** y devuelve la versión segura. `nh3.clean(value, tags=set())` elimina todas las etiquetas HTML (`tags=set()` significa "ninguna etiqueta permitida"), y además borra por completo el contenido de las peligrosas como `<script>`. Por ejemplo, `<script>alert(1)</script>Hola <b>mundo</b>` se guarda como `Hola mundo`.

Prueben creando una tarea con esa descripción y miren qué quedó guardado.

3.3 Logging y auditoría de seguridad

Concepto: qué registrar y qué no. Un registro de auditoría responde, para cada evento relevante, cinco preguntas: **quién** (usuario), **qué** (acción), **cuándo** (fecha y hora), **desde dónde** (IP) y con qué **resultado**. Eventos típicos a registrar: logins fallidos y exitosos, cambios de contraseña o de roles, accesos denegados, operaciones administrativas.

Lo que **nunca** se registra: contraseñas (ni siquiera las incorrectas: suelen ser la contraseña real con un error de tipeo), tokens, claves de API ni datos personales sensibles. Un log con secretos convierte cualquier filtración de logs en una filtración de cuentas.

Los mensajes se clasifican por **nivel**, de menor a mayor gravedad:

Nivel	Para qué	Ejemplo
`DEBUG`	Detalles para depurar, solo en desarrollo	Valor de una variable

`INFO`	Funcionamiento normal	"Tarea creada"
`WARNING`	Algo inesperado, pero el sistema sigue	Login fallido
`ERROR`	Falló una operación	No se pudo guardar
`CRITICAL`	El sistema no puede seguir	Sin conexión a la base

Hay dos usos distintos del logging: los logs **técnicos** (para depurar y monitorear, Clase 12) y los registros de **auditoría** (para seguridad y responsabilidad). Hoy guardamos la auditoría en dos lugares a propósito: en la base de datos (`SecurityLog`, fácil de consultar desde el `/admin/`) y en un archivo (`security.log`, que sigue existiendo aunque la base se comprometa).

Práctica.

Creen un modelo simple para registrar eventos sensibles. Al final de `taskflow/core/models.py`:

```
class SecurityLog(models.Model):
    user = models.ForeignKey(settings.AUTH_USER_MODEL, null=True, blank=True, on_delete=models.SET_NULL)
    action = models.CharField(max_length=100)  # ej: "login_failed", "permission_denied"
    detail = models.CharField(max_length=255, blank=True)
    ip_address = models.GenericIPAddressField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f"{self.created_at:%Y-%m-%d %H:%M} {self.action} ({self.ip_address})"
```

- `on_delete=models.SET_NULL` (a diferencia del `CASCADE` de otros modelos) significa "si se borra el usuario, dejá el registro de auditoría pero con `user` vacío": no queremos perder el historial de seguridad solo porque se eliminó una cuenta.
- `detail` guarda información adicional en texto libre (por ejemplo, qué usuario se intentó usar).
- `GenericIPAddressField` valida que el valor sea una dirección IP (v4 o v6).
- `{self.created_at:%Y-%m-%d %H:%M}` formatea la fecha dentro del f-string, para que el admin muestre algo legible.

```
python manage.py makemigrations core
python manage.py migrate
```

Registren `SecurityLog` en `taskflow/core/admin.py`, con el mismo patrón que los modelos anteriores (agréguenlo al import y sumen `admin.site.register(SecurityLog)`).

Además de la base de datos, configuramos el módulo `logging` de Python (es parte de la librería estándar, no es específico de Django) para que quede también un archivo de texto plano. Al final de `taskflow/config/settings.py`:

```
LOGGING = {
    "version": 1,
    "disable_existing_loggers": False,
    "handlers": {
        "file": {"class": "logging.FileHandler", "filename": BASE_DIR / "security.log"},
    },
    "loggers": {
        "security": {"handlers": ["file"], "level": "WARNING"},
    },
}
```

- Un **logger** es un canal con nombre al que el código le manda mensajes (`logging.getLogger("security")` en la vista de la Parte 2.3).
- Un **handler** decide a dónde van esos mensajes: acá, a un archivo.
- `BASE_DIR / "security.log"` crea el archivo en la raíz del proyecto (`taskflow/security.log`), sin importar desde qué carpeta se lance el servidor.

- `"level": "WARNING"` hace que se guarden los mensajes de nivel `warning` o más graves, e ignore los informativos.

Agreguen `security.log` a `taskflow/.gitignore`: es un archivo generado en cada máquina, no corresponde versionarlo.

Ahora sí, la prueba de las Partes 2 y 3.3. Reinicien el servidor y hagan 6 intentos de login seguidos con una contraseña incorrecta:

```
curl -i -X POST http://127.0.0.1:8000/api/token/ \
  -H "Content-Type: application/json" \
  -d '{"username": "admin", "password": "incorrecta"}'
```

Los primeros 5 devuelven `401 Unauthorized`; el sexto, dentro del mismo minuto, devuelve `429 Too Many Requests`. Después, verifiquen:

- En `/admin/` → **Security logs**: 5 registros `login_failed` con su IP.
- En `taskflow/security.log`: 5 líneas `Login fallido para usuario admin desde 127.0.0.1`.

¿Por qué 5 registros y no 6? Porque el sexto intento lo frenó el throttle **antes** de llegar a verificar la contraseña.

3.4 Variables de entorno y secretos

Concepto: gestión de secretos. Un **secreto** es cualquier dato que da acceso a algo: la `SECRET_KEY`, contraseñas de bases de datos, claves de API, credenciales de OAuth. Las reglas generales, en cualquier stack:

- **Separar la configuración del código:** el código se comparte; los secretos, no. Es uno de los principios de la metodología conocida como *The Twelve-Factor App*, muy difundida en el desarrollo de aplicaciones para la nube.
- **Un secreto distinto por ambiente:** desarrollo, pruebas y producción nunca comparten claves.
- **Mínimo privilegio también para las claves:** una clave de API debería poder hacer solo lo que la aplicación necesita.
- **Rotación:** si un secreto se filtra (por ejemplo, se subió a GitHub), no alcanza con borrarlo del repositorio, porque sigue en el historial de Git: hay que **generar uno nuevo** e invalidar el viejo.
- En producción, los secretos se cargan en el panel de la plataforma o en un gestor de secretos, nunca en archivos dentro del servidor.

Práctica.

Repasen el `.env` de la Clase 1: la `SECRET_KEY`, y ahora también las claves de OAuth de la Clase 7 (`GOOGLE_CLIENT_ID`/`GOOGLE_CLIENT_SECRET`), **siempre** van en `taskflow/.env`, nunca escritas directamente en `settings.py` ni subidas al repositorio. Confirmen, abriendo `taskflow/.gitignore`, que `.env` siga listado ahí desde la Clase 1.

Sumamos una configuración más al `.env`, pensando en el despliegue de la Clase 12. En `settings.py`, reemplacen la línea `ALLOWED_HOSTS = []` por:

```
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "127.0.0.1,localhost").split(",")
```

`ALLOWED_HOSTS` es la lista de dominios desde los que Django acepta peticiones. Con `DEBUG=True` Django es permisivo, pero con `DEBUG=False` (producción) rechaza con `400` cualquier dominio que no esté en esta lista: es una protección contra ataques que falsifican el header `Host`. Leerla del entorno permite usar valores distintos en cada ambiente sin tocar el código. `.split(",")` convierte el texto `"127.0.0.1,localhost"` en la lista `["127.0.0.1", "localhost"]`.

3.5 Concepto: encabezados de seguridad y configuración para producción

El programa incluye la "configuración de políticas de seguridad". Además de CORS, una aplicación web envía **encabezados de seguridad** que le indican al navegador cómo protegerse. `SecurityMiddleware`, que Django ya tiene activo, se encarga de varios de ellos:

Encabezado	Qué hace	En Django

`Strict-Transport-Security` (HSTS)	Obliga al navegador a usar siempre HTTPS con ese sitio	`SECURE_HSTS_SECONDS`
`X-Content-Type-Options: nosniff`	Impide que el navegador "adivine" el tipo de un archivo	Activo por defecto
`X-Frame-Options: DENY`	Impide que otro sitio muestre el nuestro dentro de un marco (ataque de *clickjacking*)	`XFrameOptionsMiddleware`, activo por defecto
`Referrer-Policy`	Controla cuánta información de la URL se envía a otros sitios	`SECURE_REFERRER_POLICY`
`Content-Security-Policy`	Define de dónde puede cargar scripts una página; es la defensa más fuerte contra XSS	`ContentSecurityPolicyMiddleware` y `SECURE_CSP` (incluidos desde Django 6.0)

Otras configuraciones que solo tienen sentido con HTTPS, y que se activan al desplegar: `SESSION_COOKIE_SECURE` y `CSRF_COOKIE_SECURE` (las cookies solo viajan cifradas) y `SECURE_SSL_REDIRECT` (redirige de HTTP a HTTPS; en plataformas como Vercel o Render la redirección ya la hace la plataforma). El comando `python manage.py check --deploy` (Clase 12) revisa todas estas opciones y avisa cuáles faltan.

Parte 4 — Guardar el avance en Git y GitHub (20 min)

```
git status
git add .
git commit -m "Agrega rate limiting, politica CORS, sanitizacion de inputs y auditoria de seguridad"
git push
```

Verifiquen en `git status` que no aparezcan ni `.env` ni `security.log`.

Parte 5 — Cierre y tarea

5.1 Resumen de la Unidad 3

- OWASP Top 10 aplicado a decisiones concretas de diseño ya tomadas en clases anteriores.
- Rate limiting general (`AnonRateThrottle`/`UserRateThrottle`) y específico para endpoints sensibles (login).
- CORS restringido a orígenes conocidos, nunca abierto a todos.
- Sanitización de inputs con `nh3`, y logging/auditoría de eventos de seguridad (`SecurityLog` + módulo `logging`).
- `ALLOWED_HOSTS` leído del entorno.

5.2 Tarea para la casa

1. Registren en `SecurityLog` (con `action="permission_denied"` y el usuario en el campo `user`) cada vez que un **miembro** reciba un `403` al intentar modificar una tarea o un proyecto. Pueden hacerlo en `IsProjectAdminOrReadOnly` (Clase 7), justo antes de devolver `False` en el caso de escritura. Pregunta para pensar: ¿por qué un usuario que **no** es miembro nunca llega a ese punto? (Pista: Clase 7, Parte 2.4, el `404`.)
2. Investiguen la diferencia entre `CORS_ALLOWED_ORIGINS` y `CORS_ALLOW_CREDENTIALS`: ¿en qué escenario se necesita esta última?
3. Guarden el avance repitiendo el flujo de la Parte 4.

5.3 Próxima clase — Unidad 4

Clase 9: dejamos el mundo request/response y entramos a **tiempo real** con Django Channels y WebSockets.

Errores frecuentes

- **El sexto login no da 429**: el servidor se reinició entre intentos (el contador vive en memoria), o `/api/token/` sigue apuntando a `TokenObtainPairView` en vez de `ThrottledTokenObtainPairView`.

- **`ImproperlyConfigured: No default throttle rate set for 'login' scope`**: falta la entrada `"login"` en `DEFAULT_THROTTLE_RATES`.
- **`NameError: name 'SecurityLog' is not defined`** al intentar loguearse: falta agregar `SecurityLog` al import de modelos en `views.py`.
- **`no such table: core_securitylog`**: se olvidaron de `makemigrations` y `migrate`.
- **Desapareció el login de Google (Clase 7)**: al editar `MIDDLEWARE` reemplazaron la lista entera y se perdió el `MIDDLEWARE += [...]` de `allauth`. Revisen el final de `settings.py`.

Preguntas de repaso

1. ¿Qué es un modelo de amenazas? Identifiquen tres activos de TaskFlow y su superficie de ataque.
2. Expliquen la defensa en profundidad usando las capas de TaskFlow. ¿Qué pasa si una capa falla?
3. ¿Por qué el ORM previene la inyección SQL? ¿Qué es una consulta parametrizada?
4. Comparen fuerza bruta, relleno de credenciales y denegación de servicio. ¿Cómo ayuda el rate limiting en cada caso?
5. ¿Qué algoritmo usa DRF para limitar peticiones? ¿Qué código y qué header devuelve al superar el límite?
6. ¿Por qué no conviene bloquear una cuenta tras varios intentos fallidos?
7. ¿Qué es la política del mismo origen? ¿Cuáles de estos pares son del mismo origen: `http://a.com` y `https://a.com`; `http://a.com:80` y `http://a.com`?
8. ¿Qué es una petición previa (preflight) y cuándo la hace el navegador?
9. ¿En qué se diferencian CORS y CSRF? ¿Por qué los endpoints con JWT no son vulnerables a CSRF?
10. Expliquen los tres tipos de XSS y las tres defensas (validar, sanitizar, escapar).
11. ¿Qué debe registrar un log de auditoría y qué no debe registrar nunca?
12. ¿Qué hay que hacer si una clave secreta se sube por error a GitHub?
13. Nombren tres encabezados de seguridad y qué protegen.

Para profundizar

- OWASP Top 10: `owasp.org/Top10`
- OWASP Cheat Sheet Series (autenticación, XSS, CSRF, logging, secretos): `cheatsheetseries.owasp.org`
- MDN, *CORS*: `developer.mozilla.org/es/docs/Web/HTTP/CORS`
- Documentación de Django sobre seguridad: `docs.djangoproject.com/en/stable/topics/security/`
- *The Twelve-Factor App*: `12factor.net/es/`

Diccionario de términos de esta clase

- **Activo:** todo lo que tiene valor y hay que proteger (datos, cuentas, claves, disponibilidad).
- **Auditoría:** registro sistemático de eventos relevantes de seguridad, para poder revisarlos después.
- **CORS (Cross-Origin Resource Sharing):** mecanismo del navegador que restringe qué orígenes pueden hacer peticiones JavaScript hacia una API.
- **CSRF:** ataque que hace que el navegador del usuario envíe una petición no deseada a un sitio donde tiene la sesión abierta.
- **Defensa en profundidad:** combinar varias capas de seguridad independientes.
- **Encabezados de seguridad:** headers HTTP que le indican al navegador cómo proteger una página (HSTS, X-Frame-Options, CSP).
- **Escapar (codificar la salida):** convertir caracteres especiales para que el navegador los muestre como texto y no los ejecute.
- **Fuerza bruta:** ataque que consiste en probar muchas combinaciones (por ejemplo, de contraseña) hasta acertar.
- **Handler (logging):** componente que decide a dónde se envían los mensajes de log (archivo, consola, servicio externo).
- **Inyección:** ataque en el que un dato del usuario termina ejecutándose como código (SQL, comandos, prompts).

- **Middleware:** componente que procesa cada petición y respuesta de forma transversal, antes o después de que llegue a la vista.
- **Origen (origin):** combinación de protocolo, dominio y puerto que identifica de dónde proviene una petición web.
- **OWASP:** fundación sin fines de lucro dedicada a la seguridad de aplicaciones web; publica el "OWASP Top 10", gratuito y de acceso público.
- **Petición previa (preflight):** petición `OPTIONS` que hace el navegador para preguntar si una petición entre orígenes está permitida.
- **Política del mismo origen:** regla del navegador que impide que una página lea respuestas de otros orígenes.
- **Rate limiting / Throttling:** limitar la cantidad de peticiones que un cliente puede hacer en un período de tiempo.
- **Relleno de credenciales:** ataque que prueba combinaciones de usuario y contraseña filtradas de otros sitios.
- **Sanitización:** proceso de limpiar una entrada de datos para eliminar contenido potencialmente peligroso (por ejemplo, HTML/JS malicioso).
- **Superficie de ataque:** conjunto de puntos por donde un atacante puede interactuar con el sistema.
- **XSS (Cross-Site Scripting):** ataque que consiste en guardar código JavaScript en datos que después se muestran en el navegador de otros usuarios.